COMPARATIVE ANALYSIS OF KERAS AND PYTORCH FRAMEWORKS IN TRANSFER-LEARNING-BASED IMAGE CLASSIFICATION: A REPRODUCIBLE BENCHMARK STUDY

YUMING HUANG[1], XIMING YANG[1], FATIMAH BINTI KHALID[1,*]

[1]Faculty of Computer Science and Information Technology,
Universiti Putra Malaysia, Malaysia

E-mail: 111@gmail.com; 111@gmail.com; 111@gmail.com

ABSTRACT

Deep-learning frameworks such as Keras (TensorFlow backend) and PyTorch are widely adopted for image classification via transfer learning, yet few studies provide reproducible, head-to-head comparisons with aligned hyperparameters, identical data splits, and equivalent training protocols. The assumption that framework choice is performance-neutral—that identical architectures with matched hyperparameters yield equivalent classification outcomes—has not been rigorously validated, leaving practitioners without evidence-based guidance on framework selection for deployment-critical applications. This study systematically benchmarks Keras and PyTorch under strictly controlled conditions to determine whether framework choice introduces statistically significant differences in classification accuracy, macro-F1, and training throughput. Three ImageNet-pretrained architectures (ResNet50, VGG16-BN, MobileNetV2) are fine-tuned on two datasets—PlantVillage (38-class plant disease, 224×224) and CIFAR-10 (10-class general objects, $32 \times 32 \rightarrow 224 \times 224$)—using aligned SGD optimizers, cosine-annealing learning-rate schedules with linear warmup, identical 3-fold stratified cross-validation, and early stopping. A total of 36 training runs (2 frameworks $\times$ 3 models $\times$ 3 folds $\times$ 2 datasets) are executed. On PlantVillage, PyTorch achieved 99.77% (ResNet50), 99.76% (VGG16), and 99.74% (MobileNetV2) test accuracy, while Keras achieved comparable results for ResNet50 (99.47%) and VGG16 (99.30%) but suffered catastrophic failure with MobileNetV2 (57.18%). On CIFAR-10, Keras ResNet50 (95.85%) exhibited the highest stability, PyTorch VGG16 (95.73%) achieved the highest accuracy, and Keras MobileNetV2 again collapsed (39.61% vs. PyTorch's 95.39%). Of 18 paired t-tests, 15 reached significance at $\alpha = 0.05$, but practical differences excluding MobileNetV2 were below 2.5 percentage points. The findings indicate that framework equivalence is conditionally dependent on the (architecture, dataset, hyperparameter) triple: for heavy architectures (ResNet50, VGG16), framework-induced accuracy differences are statistically significant but practically marginal (< 2.5 pp), whereas for lightweight architectures (MobileNetV2), the Keras implementation exhibited catastrophic training instability across both datasets, demonstrating that framework choice can be a critical deployment variable for specific model families.

Keywords: *Transfer Learning, Image Classification, Deep Learning, Keras, PyTorch, PlantVillage, CIFAR-10, Framework Comparison, Reproducibility.*

1 INTRODUCTION

Image classification is a cornerstone task in computer vision with broad applications spanning agriculture, medical imaging, autonomous driving, and industrial inspection. In agricultural contexts, automated identification of crop diseases from leaf images has the potential to reduce diagnostic time and improve intervention strategies. Transfer learning from large-scale pretrained models—typically those trained on the ImageNet dataset [1]—has become the standard paradigm for achieving high classification accuracy with limited domain-specific labelled data [2]. By leveraging hierarchical feature representations learned from over one million natural images, practitioners can fine-tune deep convolutional neural networks (CNNs) on downstream tasks with substantially fewer training samples and reduced computational cost compared to training from scratch.

Two deep-learning frameworks dominate the current ecosystem: TensorFlow/Keras and PyTorch. Keras, serving as a high-level API atop TensorFlow, offers rapid prototyping capabilities

and a declarative model-building interface that has made it widely popular among applied researchers and industry practitioners [3]. PyTorch, developed by Meta AI, provides a flexible imperative paradigm with dynamic computation graphs that has become the predominant framework in academic research [4]. Despite their widespread adoption, practitioners often assume that framework choice is neutral—that the same architecture with the same hyperparameters produces identical results regardless of the underlying implementation. This assumption, however, has rarely been rigorously validated under controlled experimental conditions.

Existing comparative studies between deep-learning frameworks suffer from several methodological limitations. Some studies compare frameworks at the API-convenience or training-speed level without controlling hyperparameters across implementations [12]. Others focus on a single dataset or a single model architecture and do not generalise across domains or model families. There is a notable absence of large-scale, strictly controlled benchmarks that align every training detail—optimizer configuration, learning-rate schedule, data augmentation pipeline, data splits, and random seeds—across frameworks. Without such alignment, observed performance differences may be attributable to incidental implementation choices rather than fundamental framework-level behaviour.

Formally, let $D = \{(x_i, y_i)\}_{i=1}^{N}$ denote a labelled image dataset where $x_i \in \mathbb{R}^{H \times W \times 3}$ represents an RGB image and $y_i \in \{1, \ldots, C\}$ denotes the corresponding class label. The task is to train a classifier $f_\theta : \mathbb{R}^{H \times W \times 3} \to \mathbb{R}^C$ via transfer learning from ImageNet-pretrained weights, fine-tuning all layers using SGD. Two implementations of f_θ are constructed—one in Keras (TensorFlow backend) and one in PyTorch—with identical architecture definitions, loss functions, optimizer configurations, learning-rate schedules, data augmentation pipelines, and cross-validation splits. The study is guided by three research questions. **RQ1:** Do the Keras and PyTorch implementations exhibit statistically significant differences in classification accuracy and macro-F1 score? **RQ2:** Do the two frameworks differ significantly in training throughput (images per second)? **RQ3:** Do the training dynamics—specifically convergence speed and early-stopping epoch—vary systematically between frameworks? For each question, the null hypothesis H_0 states that there is no significant difference in the target metric between frameworks across folds, for a given (model, dataset) pair, at significance level $\alpha = 0.05$.

This study addresses the identified gap through three main contributions. First, a reproducible, fully aligned benchmark is presented, comparing Keras and PyTorch across three architectures (ResNet50 [5], VGG16-BN [6], MobileNetV2 [7]), two datasets (PlantVillage [8] and CIFAR-10 [10]), and three cross-validation folds, yielding 36 training runs with every hyperparameter, data split, and augmentation matched. Second, a rigorous statistical analysis employing paired t-tests, Cohen's d, and Cliff's delta is conducted to quantify whether framework choice introduces significant differences in accuracy, macro-F1, or training throughput. Third, complete experiment artifacts—including code, configuration files, training logs, and model checkpoints—are released to enable full reproducibility.

The remainder of this paper is organised as follows. Section 2 reviews related work. Section 3 presents the proposed methodology. Section 4 describes the experimental setup. Section 5 reports results and discussion. Section 6 concludes the paper and outlines future work.

2 RELATED WORK

2.1 Transfer Learning for Image Classification

Transfer learning has fundamentally transformed the practice of image classification by enabling the reuse of feature representations learned from large-scale source domains. The seminal work by Deng et al. [1] introduced the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), providing a dataset of over 1.2 million images spanning 1,000 categories that has since served as the de facto pretraining source for transfer learning. Yosinski et al. [11] subsequently demonstrated that features learned in the lower layers of deep neural networks are general and transferable across tasks, while higher layers become increasingly task-specific. This finding established two dominant transfer-learning paradigms: feature extraction, in which pretrained convolutional layers are frozen and only the classification head is retrained, and full fine-tuning, in which all layers are updated with a reduced learning rate. A comprehensive survey by Zhuang et al. [2] further categorised transfer-learning strategies and confirmed that full fine-tuning generally yields superior performance when sufficient target-domain data is available, as it allows the pretrained representations to adapt to the statistical properties of the target distribution.

The architectures employed in transfer learning have evolved substantially since the early AlexNet-based studies. He et al. [5] introduced ResNet, which employed skip connections to enable training of networks exceeding 100 layers and achieved state-of-the-art performance on ImageNet. Simonyan and Zisserman

[6] proposed VGGNet, demonstrating that deep networks with small (3×3) convolutional filters could achieve competitive accuracy. Sandler et al. [7] designed MobileNetV2 with inverted residual blocks and depthwise separable convolutions, reducing model size by an order of magnitude while maintaining competitive accuracy—making it suitable for mobile and edge deployment. These three architectures—ResNet50, VGG16, and MobileNetV2—span a representative range of model complexity (2.3M to 134M parameters) and architectural design philosophies, making them appropriate candidates for a cross-framework benchmark.

In the domain of plant disease recognition, the PlantVillage dataset introduced by Hughes and Salathé [8] has become a standard benchmark, containing over 54,000 images of healthy and diseased plant leaves across 38 classes. Mohanty et al. [9] demonstrated that deep CNNs fine-tuned on PlantVillage achieved classification accuracies exceeding 99%, establishing the viability of automated disease diagnostics from leaf images. Subsequent studies have explored various architectures for this task, including ensemble approaches and hybrid pipelines that combine CNN-extracted features with classical machine-learning classifiers such as SVM and logistic regression [17, 18]. These hybrid approaches offer computational efficiency advantages but generally underperform end-to-end fine-tuned models on large-scale benchmarks.

CIFAR-10 [10], comprising 60,000 images across 10 general-object categories at 32×32 resolution, serves as a complementary benchmark representing low-resolution general-purpose classification. Although originally designed for training-from-scratch evaluation, CIFAR-10 has been increasingly used in transfer-learning studies where images are upsampled to match the input resolution expected by ImageNet-pretrained models (224×224). This upsampling introduces bilinear interpolation artefacts and a distribution shift from the natural-image statistics on which ImageNet models were pretrained, making CIFAR-10 a challenging test case for transfer-learning robustness. The inclusion of both PlantVillage and CIFAR-10 in this benchmark enables evaluation across two fundamentally different data regimes: high-resolution domain-specific imagery and low-resolution upsampled general-purpose imagery.

2.2 Framework Comparisons in Deep Learning

The deep-learning framework landscape has undergone rapid evolution since the introduction of Theano [3]. TensorFlow, released by Google in 2015, introduced a static computation graph paradigm and has since evolved to integrate Keras as its official high-level API, offering a streamlined interface for model construction and training. PyTorch, released by Meta AI in 2017 [4], adopted an imperative, define-by-run paradigm with dynamic computation graphs that provides greater debugging flexibility and has become the dominant framework in academic research. As of 2024, PyTorch accounts for the majority of publications at top machine-learning venues, while TensorFlow/Keras maintains a strong presence in industry deployments [13].

Several studies have compared deep-learning frameworks along multiple dimensions. Shi et al. [12] benchmarked TensorFlow, Caffe, and Torch on standard CNN architectures including AlexNet, GoogLeNet, and VGG, focusing on training speed and GPU utilisation. Their results indicated that performance differences were architecture-dependent and that no single framework was universally superior in terms of throughput. More recent comparisons between TensorFlow/Keras and PyTorch have examined developer productivity, debugging experience, and deployment ecosystem integration [13]. However, these studies primarily address software-engineering and usability considerations rather than the scientific question of whether framework choice affects classification accuracy under controlled conditions.

A critical gap in the existing literature is the absence of hyperparameter-aligned accuracy comparisons with statistical rigour. Most framework comparisons either use each framework's default hyperparameters—which differ in subtle but consequential ways (e.g., BatchNorm momentum conventions, weight-decay implementation, gradient clipping defaults)—or focus exclusively on training throughput without controlling for model convergence quality. When classification accuracy is reported, it is typically based on a single train/test split without cross-validation or significance testing, making it impossible to determine whether observed differences are systematic or due to random variation. Furthermore, no prior study has systematically investigated whether framework choice interacts with architecture complexity—i.e., whether lightweight models (MobileNetV2) exhibit different cross-framework behaviour than heavyweight models (ResNet50, VGG16). This study addresses these gaps by enforcing strict hyperparameter alignment, employing three architectures of varying complexity, evaluating on two datasets with different characteristics, and applying paired statistical tests across multiple folds.

2.3 Experimental Reproducibility in Deep Learning

Reproducibility in deep-learning research faces several well-documented challenges that are particularly relevant to cross-framework comparisons. Non-determinism arising from cuDNN autotuning, floating-point arithmetic order, and GPU parallelism can introduce run-to-run variance even with fixed random seeds [15]. Pham et al. [15] systematically investigated sources of non-determinism in deep-learning training and found that even on the same hardware with the same code, training runs can diverge substantially due to non-deterministic GPU operations. Data shuffling order, weight initialisation schemes, and framework-specific implementation details (e.g., BatchNorm momentum conventions, weight-decay application methods, mixed-precision scaling strategies) further compound the reproducibility challenge.

The importance of rigorous statistical evaluation in machine-learning research has been increasingly emphasised. Henderson et al. [14] demonstrated that in reinforcement learning, reported performance improvements often fall within the range of run-to-run variance when evaluated across multiple random seeds, highlighting that many published claims of superiority may be artefacts of insufficient statistical testing. This concern extends directly to supervised learning and framework comparisons: without cross-validation and paired statistical tests, small differences between frameworks may be erroneously interpreted as meaningful or, conversely, real differences may be dismissed as noise. Bouthillier et al. [16] further argued that sources of variation in deep learning extend beyond random seeds to include data splits, data augmentation order, and hardware-specific numerical behaviour, reinforcing the need for multi-fold evaluation protocols.

The present study addresses each of these reproducibility concerns through a comprehensive set of mechanisms. Random seeds are fixed per fold using the formula $\text{seed} = 42 + \text{fold_index}$, ensuring that both frameworks process identical random states within each fold. Stratified splits guarantee class-proportional representation in all training, validation, and test partitions. Per-fold channel-wise normalisation statistics (mean and standard deviation per colour channel) are precomputed exclusively from the training partition and stored in configuration files, preventing information leakage from validation or test data. All training configurations are centralised in a single YAML file shared by both framework implementations. Epoch-level training logs, test-time evaluation metrics, and confusion matrices are archived for all 36 runs, enabling post-hoc verification of every reported number. Finally, both Keras and PyTorch implementations share the same data-loading indices, augmentation specifications, and evaluation protocol, isolating framework-level differences from all other experimental variables.

2.4 Summary of Related Work

Prior framework comparisons generally lack hyperparameter alignment across implementations, employ only a single dataset or model architecture, and do not apply statistical significance testing to performance differences. Several studies that do employ statistical testing (e.g., Henderson et al. [14]) operate in non-supervised-learning domains or do not compare frameworks directly. The present study is, to the best of the authors' knowledge, the first to combine strict hyperparameter alignment, multi-architecture and multi-dataset evaluation, stratified cross-validation, and paired statistical testing with effect-size reporting in a single Keras–PyTorch comparison for image classification.

3 PROPOSED METHODOLOGY

3.1 Overview of the Benchmark Pipeline

The proposed benchmark follows a six-stage pipeline: **Data Preparation** → **Fold Splitting** → **Training (Keras / PyTorch)** → **Evaluation** → **Aggregation** → **Statistical Testing**. Every component in this pipeline is made identical across the two framework implementations—including dataset loading, index-based splitting, augmentation specifications, optimizer configuration, learning-rate scheduling, and evaluation metrics—so that the only variable under investigation is the framework-internal execution. Figure 1 illustrates the end-to-end pipeline architecture.

[Pipeline overview diagram placeholder]
Data → Splits → Training → Eval → Aggregation → Stats

Figure 1: End-to-end benchmark pipeline. All components except the framework-internal training loop are shared between Keras and PyTorch.

3.2 Datasets

PlantVillage. The PlantVillage dataset, sourced from Kaggle, contains 38 classes of plant leaf im-

ages covering both healthy and diseased conditions [8]. All images were resized to $224 \times 224 \times 3$ and stored as uint8 NumPy arrays, yielding approximately 54,000 usable images. Class imbalance exists across the 38 categories; this is handled through stratified splitting to ensure proportional class representation in all partitions.

CIFAR-10. The standard CIFAR-10 dataset was obtained via `tf.keras.datasets.cifar10` [10]. It contains 60,000 images of size $32 \times 32 \times 3$ distributed equally across 10 general-object categories. Images are stored at their original 32×32 resolution and resized to 224×224 at training time through the data pipeline to match the input resolution expected by ImageNet-pretrained models. This upsampling via bilinear interpolation introduces artefacts that represent a controlled distribution shift from the ImageNet pretraining domain. Both datasets employ standard data augmentation techniques—random resized cropping and horizontal flipping—which have been shown to improve generalisation and reduce overfitting in CNN-based image classification [19].

3.3 Data Splitting and Cross-Validation Protocol

A two-stage splitting procedure was employed using scikit-learn [20]. First, 20% of each dataset was held out as a fixed test set using stratified random sampling (`train_test_split` with `stratify=y`, `test_size=0.2`, `random_state=42`). The remaining 80% formed the trainval subset, which was partitioned into three folds using `StratifiedKFold` (`n_splits=3`, `shuffle=True`, `random_state=42`). Fold indices are relative to the trainval subset; a two-stage indexing mechanism maps fold-local indices to global dataset indices to ensure consistency across framework implementations.

Per-fold normalisation statistics were computed exclusively from each fold's training partition to prevent information leakage. Channel-wise mean and standard deviation values were precomputed and stored in configuration files. For reference, PlantVillage fold 0 yielded mean $= [0.4668, 0.4893, 0.4106]$ and std $= [0.1955, 0.1706, 0.2137]$, while CIFAR-10 fold 0 yielded mean $= [0.4916, 0.4824, 0.4471]$ and std $= [0.2470, 0.2433, 0.2615]$.

3.4 Model Architectures

Three ImageNet-pretrained architectures spanning a wide range of model complexity were employed. Table 1 summarises the parameter counts for each configuration.

ResNet50. In PyTorch, the model was loaded via `torchvision.models.resnet50` with `IMAGENET1K_V1` weights; the final fully connected layer was replaced with a `Linear(2048, num_classes)` layer using Kaiming normal initialisation. In Keras, the model was constructed using `tf.keras.applications.ResNet50` with `include_top=False`, `weights='imagenet'`, and `pooling='avg'`, followed by a `Dense(num_classes)` layer with L2 kernel regularisation. Parameter counts differed by 0.11% between frameworks (23,585,894 vs. 23,612,454 for PlantVillage) due to differences in how the global average pooling and classification head are parametrised.

VGG16-BN. In PyTorch, `torchvision.models.vgg16_bn` with `IMAGENET1K_V1` weights was used; the final classifier layer was replaced to match the target number of classes. Batch Normalisation [21] is applied after each convolutional layer to stabilise training and accelerate convergence. In Keras, VGG16-BN was *manually implemented* using `Conv2D + BatchNormalization + ReLU` per block, because `tf.keras.applications.VGG16` does not include BatchNorm layers. Convolutional and dense weights were copied from the official VGG16 pretrained model; BatchNorm layers were initialised to defaults (mean= 0, var= 1). This introduces an inherent alignment limitation: PyTorch loads ImageNet-trained BN running statistics, whereas Keras BN starts from default values. This asymmetry is disclosed as a known confound and is discussed in Section 5. Both implementations yielded 134,424,678 parameters (0.0% difference).

MobileNetV2. MobileNetV2 is the lightest architecture in this benchmark, building upon the depthwise separable convolution paradigm introduced in the original MobileNet [22] and extending it with inverted residual blocks to achieve competitive accuracy at a fraction of the parameter cost. In PyTorch, the model was loaded via `torchvision.models.mobilenet_v2` with `IMAGENET1K_V1` weights; the classifier head was replaced with a new `Linear(1280, num_classes)` layer. In Keras, the model was constructed using `tf.keras.applications.MobileNetV2` with `include_top=False`, `weights='imagenet'`, and `pooling='avg'`, followed by a new `Dense(num_classes)` layer with L2 kernel regularisation. Unlike VGG16-BN, both framework implementations load official ImageNet-pretrained weights including BatchNorm running statistics, so no BN initialisation asymmetry exists for this architecture. Both implementations yielded identical parameter counts of 2,272,550 (0.0% difference).

Table 1: Parameter Counts by Model and Framework (PlantVillage, 38 Classes)

Model	PyTorch	Keras	Diff (%)
ResNet50	23,585,894	23,612,454	0.11
VGG16-BN	134,424,678	134,424,678	0.00
MobileNetV2	2,272,550	2,272,550	0.00

3.5 Training Configuration

All hyperparameters were strictly aligned across frameworks, as summarised in Table 2.

Table 2: Aligned Hyperparameter Configuration

Hyperparameter	Value	Notes
Optimizer	SGD	Both frameworks
Momentum	0.9	Nesterov enabled
Weight decay	1×10^{-4}	See note below
Base LR	0.04	Batch 128 scaling
LR schedule	Cosine anneal.	$\eta_{\min}=0$
Warmup	5 epochs	Linear per-step
Batch size	128	—
Max epochs	30	—
Early stopping	patience=7	On val_loss
Loss	Cross-entropy	From logits
Input size	224×224	CIFAR-10 resized
Augmentation	RRC + HFlip	scale $[0.8, 1.0]$
Random seed	42+fold	Per-fold

Weight decay implementation. PyTorch applies weight decay directly within the SGD optimizer via the `weight_decay` parameter. Keras lacks a native weight-decay parameter in its SGD implementation; instead, `kernel_regularizer=l2(weight_decay/2)` is applied to each layer, and the resulting regularisation losses (`model.losses`) are summed with the classification loss inside the `GradientTape` loop. Both approaches yield mathematically equivalent gradient updates: $g \leftarrow g + \lambda \cdot w$, where λ denotes the weight-decay coefficient.

Learning-rate schedule. A custom `WarmupCosineLRSchedule` class was implemented identically in both frameworks, following the cosine annealing strategy proposed by Loshchilov and Hutter [23]. During the first warmup_epochs $\times$ steps_per_epoch steps, the learning rate increases linearly from 0 to the base learning rate (0.04), adopting the gradual warmup strategy advocated by Goyal et al. [24] for training with large batch sizes. For the remaining steps, cosine annealing decays the learning rate to $\eta_{\min} = 0$. Critically, the learning rate is updated *per step* (not per epoch) in both frameworks, ensuring identical LR trajectories at every training iteration.

3.6 Training Loop Alignment

The training loops were designed to be structurally equivalent across frameworks. In PyTorch, the standard loop follows: `optimizer.zero_grad()`, forward pass, `loss.backward()`, `optimizer.step()`, with automatic mixed precision (AMP) enabled via `torch.cuda.amp` following the methodology described by Micikevicius et al. [25]. In Keras, a custom `tf.GradientTape` loop was implemented—deliberately avoiding the high-level `model.fit()` API—to match the PyTorch loop structure step by step. Mixed precision was *not* used in the Keras implementation: initial experiments with Keras mixed precision via `LossScaleOptimizer` yielded near-random accuracy ($\sim$3%) due to incorrect loss scaling in the custom `GradientTape` loop; this was resolved by reverting to float32 training. This precision asymmetry (PyTorch AMP vs. Keras float32) is acknowledged as a known experimental limitation. The best model checkpoint for each fold was selected based on the highest validation accuracy observed during training.

3.7 Evaluation Metrics

Three primary metrics were computed for each training run. **Test accuracy** is defined as the proportion of correctly classified samples in the held-out test set. **Macro-F1** is the unweighted mean of per-class F1 scores, providing robustness to class imbalance by giving equal weight to all classes regardless of their sample counts. **Training throughput** is measured in images per second, computed as $(\lfloor N_{\text{train}}/B \rfloor \times B)/t_{\text{epoch}}$, where B is the batch size and t_{epoch} is the wall-clock time for one training epoch. All metrics are reported as 3-fold mean $\pm$ standard deviation.

3.8 Statistical Analysis Protocol

For each of the 18 (dataset, model, metric) combinations, a **paired t-test** was conducted by pairing Keras and PyTorch results by fold, yielding three paired observations per test. The null hypothesis H_0: mean difference $= 0$ was tested at significance level $\alpha = 0.05$. Two effect-size measures supplemented each test. **Cohen's** d was computed as $d = \bar{x}_{\text{diff}}/s_{\text{diff}}$, where $\bar{x}_{\text{diff}}$ and s_{diff} are the mean and standard deviation of the paired differences. **Cliff's delta** (δ), a non-parametric ordinal effect-size measure, was computed to provide a distribution-free complement to Cohen's d.

4 EXPERIMENTAL SETUP

4.1 Hardware and Software Environment

All experiments were executed on a compute node equipped with an NVIDIA A40 GPU (48 GB VRAM). The cuDNN version was 9.1.0.02 (reported via `torch.backends.cudnn.version()` = 91002). The software environment comprised Python 3.10.12, TensorFlow 2.20.0, PyTorch 2.9.1, scikit-learn, NumPy, pandas, matplotlib, and SciPy. The `environment.txt` file was recorded on a login node where NVML initialisation failed; however, actual training executed on GPU-equipped compute nodes, as confirmed by epoch-level throughput figures (60–370 images/sec) that are inconsistent with CPU-only execution.

4.2 Experiment Matrix

The full experiment comprised 2 datasets $\times$ 3 models $\times$ 2 frameworks $\times$ 3 folds = **36** training runs. Each run produced the following artifacts: an epoch-level CSV log recording loss, accuracy, learning rate, and timing per epoch; a test JSON file containing final evaluation metrics; a confusion matrix stored as a `.npy` file; and a best-model checkpoint. All artifacts follow the naming convention `{framework}_{dataset}_{model}_fold{fold}.*`

4.3 Reproducibility Artifacts

To enable full reproducibility, following the recommendations of Pineau et al. [26] on the machine-learning reproducibility checklist, the following artifacts were generated and archived: (a) `configs/train_config.yaml` containing all hyperparameters; (b) `configs/dataset_stats.yaml` storing per-fold channel-wise normalisation statistics; (c) `data/splits/` containing all train/val/test index files; (d) `logs/` containing 36 epoch-level CSVs, 36 test JSONs, and 36 confusion matrices; (e) `results/results.csv` with per-fold aggregated results (36 rows); (f) `results/results_summary.csv` with mean$\pm$std summaries (12 rows); and (g) `analysis/` containing statistical test outputs.

5 RESULTS AND DISCUSSION

5.1 Overall Classification Performance

Tables 3 and 4 present the mean test accuracy and macro-F1 scores across three folds for all 12 (dataset, model, framework) combinations.

High-resolution domain-specific tasks (PlantVillage). PlantVillage represents high-resolution, fine-grained domain-specific classification (38 classes, 224×224). ResNet50 and VGG16 demonstrated strong cross-framework stability: both frameworks achieved >99.3% accuracy, with PyTorch holding a statistically significant but practically marginal advantage of 0.30 pp (ResNet50) and 0.46 pp (VGG16). From a deployment perspective, this margin is unlikely to influence real-world decision-making; both implementations are equally suitable for production-grade pipelines. The choice between Keras and PyTorch for these architectures may therefore be guided by ecosystem factors—developer familiarity, TensorFlow Serving vs. TorchServe integration—rather than classification performance. MobileNetV2, however, presents a strikingly different picture: PyTorch achieved 99.74% (on par with heavier architectures), while Keras collapsed to 57.18%, rendering it unsuitable for any deployment context in this framework (analysed in Section 5.6).

Low-resolution upsampled tasks (CIFAR-10). CIFAR-10 represents a fundamentally different scenario: natively low-resolution images (32×32) upsampled to 224×224, introducing bilinear interpolation artefacts and distribution shift from ImageNet. Keras ResNet50 exhibited the most stable behaviour (95.85%, $\sigma = 0.10\%$), making it particularly suitable for applications requiring consistent, reproducible performance—e.g., regulatory-grade classification systems. PyTorch ResNet50, while achieving the highest individual fold accuracy (96.68% in fold 2), suffered warmup-phase instability in two out of three folds (early stopping at epoch 8 with $\sim$91.9%), producing high variance ($\sigma = 2.78\%$). PyTorch VGG16 delivered the most robust high-accuracy performance on CIFAR-10 (95.73%, $\sigma = 0.12\%$), with its 1.94 pp advantage over Keras VGG16 attributable to pretrained BatchNorm statistics (Section 5.5). Keras MobileNetV2 again collapsed catastrophically (39.61%).

Deployment-constrained scenarios (lightweight models). MobileNetV2 (2.27M parameters) is 10$\times$ smaller than ResNet50 (23.6M) and 59$\times$ smaller than VGG16 (134.4M). In PyTorch, MobileNetV2 achieved accuracy

Table 3: Mean Test Accuracy (%) across 3 Folds

Dataset	Model	Keras (mean±std)	PyTorch (mean±std)	Δ (PT−Keras)
PlantVillage	ResNet50	99.47 ± 0.02	99.77 ± 0.05	+0.30 pp
PlantVillage	VGG16	99.30 ± 0.10	99.76 ± 0.06	+0.46 pp
PlantVillage	MobileNetV2	57.18 ± 10.45	99.74 ± 0.05	**+42.56 pp**
CIFAR-10	ResNet50	95.85 ± 0.10	93.47 ± 2.78	−2.39 pp
CIFAR-10	VGG16	93.79 ± 0.23	95.73 ± 0.12	+1.94 pp
CIFAR-10	MobileNetV2	39.61 ± 8.85	95.39 ± 0.14	**+55.78 pp**

Table 4: Mean Test Macro-F1 across 3 Folds

Dataset	Model	Keras (mean±std)	PyTorch (mean±std)	Δ (PT−Keras)
PlantVillage	ResNet50	0.9931 ± 0.0001	0.9964 ± 0.0005	+0.0033
PlantVillage	VGG16	0.9907 ± 0.0014	0.9966 ± 0.0005	+0.0059
PlantVillage	MobileNetV2	0.4688 ± 0.1045	0.9963 ± 0.0004	**+0.5275**
CIFAR-10	ResNet50	0.9584 ± 0.0010	0.9345 ± 0.0279	−0.0240
CIFAR-10	VGG16	0.9377 ± 0.0023	0.9572 ± 0.0012	+0.0195
CIFAR-10	MobileNetV2	0.3757 ± 0.1032	0.9538 ± 0.0014	**+0.5781**

fully competitive with heavier architectures (99.74% on PlantVillage, 95.39% on CIFAR-10), demonstrating that parameter efficiency need not sacrifice accuracy. However, MobileNetV2's deployment viability is framework-dependent: the Keras implementation proved unreliable across both datasets (6 out of 6 runs failed). Practitioners requiring lightweight models for edge or mobile deployment should use PyTorch MobileNetV2 with ONNX export; if TensorFlow deployment is mandatory, ResNet50 or VGG16 remain more reliable, albeit at higher parameter cost.

Accuracy–F1 correlation. Across all 12 (dataset, model, framework) combinations, test accuracy and macro-F1 were highly correlated (Pearson $r > 0.999$). In the high-performing configurations (>93%), accuracy and macro-F1 differed by less than 0.3 percentage points, confirming that class-imbalance effects are well-controlled by stratified splitting. The two metrics diverge only in the catastrophically failed Keras MobileNetV2 runs, where macro-F1 (0.37–0.47) is further depressed relative to accuracy (0.40–0.57) due to per-class F1 collapse on minority classes.

5.2 Training Throughput Comparison

Table 5 presents the mean training throughput across three folds for each configuration.

Small-source-image pipelines (CIFAR-10). On CIFAR-10 (32×32 source images upsampled to 224×224), Keras was substantially faster: 1.88× for ResNet50 (214 vs. 114 images/sec), 2.09× for VGG16 (137 vs. 66 images/sec), and 2.30× for MobileNetV2 (369 vs. 161 images/sec). This advantage is attributable to `tf.data`'s fused image-processing operations (resize, crop, and flip as a single computation-graph node), which are more efficient than PyTorch's PIL-based `torchvision.transforms` pipeline when the dominant cost is image upsampling. Additionally, PyTorch's AMP overhead (GradScaler bookkeeping) provides less net speedup on a compute-light task. For throughput-sensitive applications on low-resolution data, Keras offers a significant training-speed advantage—provided the chosen architecture is not MobileNetV2. The Keras MobileNetV2 throughput figure (369 images/sec) reflects an early-stopped, failed training run (epoch 8–9) and does not represent a functionally comparable pipeline.

Native-resolution pipelines (PlantVillage). On PlantVillage (native 224×224 images), the throughput picture reverses. PyTorch ResNet50 was approximately 28% faster (124.9 vs. 97.9 images/sec), though this did not reach significance ($p = 0.290$) due to high Keras timing variance from a fold-0 anomaly (Keras ResNet50 fold 0 ran 43 epochs before early stopping vs. 30 for the other folds). VGG16 throughput was similar across frameworks (∼73–76 images/sec). On native-resolution data where GPU compute is the bottleneck rather than data loading, PyTorch's AMP and optimised DataLoader (`persistent_workers`, `pin_memory`, `prefetch_factor`) narrow or eliminate Keras's `tf.data` advantage. Neither framework is universally faster; throughput advantage is contingent on whether the pipeline is data-loading-bound or compute-bound.

Table 5: Mean Training Throughput (images/sec) across 3 Folds

Dataset	Model	Keras (mean±std)	PyTorch (mean±std)	Δ (PT−Keras)	p-value
PlantVillage	ResNet50	97.9 ± 31.9	124.9 ± 0.9	+27.0	0.290 (NS)
PlantVillage	VGG16	72.8 ± 0.1	76.3 ± 0.6	+3.5	0.008
PlantVillage	MobileNetV2	210.3 ± 23.7	179.3 ± 17.9	−31.0	0.029
CIFAR-10	ResNet50	214.0 ± 2.6	114.4 ± 18.3	−99.6	0.013
CIFAR-10	VGG16	137.1 ± 0.6	65.6 ± 8.8	−71.5	0.005
CIFAR-10	MobileNetV2	369.0 ± 0.8	160.5 ± 14.2	−208.6	0.002

5.3 Training Dynamics

Figure 2 presents the training curves (loss vs. epoch and validation accuracy vs. epoch) for each dataset–model pair, with per-fold thin lines and mean bold lines for both frameworks.

[Training curves placeholder]
2×3 grid: 2 datasets × 3 models
Per-fold thin lines + mean bold line

Figure 2: Training curves for all dataset–model pairs. Each subplot shows per-fold trajectories (thin lines) and the mean (bold line) for both Keras (blue) and PyTorch (orange).

Stable convergence cases. For ResNet50 and VGG16 on PlantVillage, both frameworks exhibited smooth, monotonic convergence. Training loss decreased continuously over 23–30 epochs; validation accuracy climbed steadily to >99%; no early stopping was triggered in 10 out of 12 runs. Keras and PyTorch training curves are nearly overlapping, confirming that the hyperparameter alignment was effective for these configurations. The per-fold variance was minimal ($\sigma < 0.1\%$), indicating robust generalisation.

PyTorch ResNet50 CIFAR-10 instability. PyTorch ResNet50 on CIFAR-10 revealed a warmup-phase instability. In folds 0 and 1, validation accuracy peaked at epoch 1 (91.97–91.98% at LR ≈ 0.008), then declined monotonically as the learning rate increased toward the base rate of 0.04. Early stopping triggered at epoch 8 in both cases. Fold 2, however, survived the warmup transition and converged normally to 96.69% at epoch 29. Keras ResNet50 showed no analogous instability—all three folds completed 30 epochs with stable convergence (95.70–96.04%). This fold-dependent behaviour suggests that the combination of AMP mixed precision, aggressive warmup LR, and distribution-shifted inputs (32 → 224 upsampling) creates a narrow stability region that is sensitive to the random initialisation of the classification head.

Keras MobileNetV2 catastrophic divergence. Keras MobileNetV2 displayed a distinctive failure pattern across all 6 runs (2 datasets × 3 folds): training accuracy rose normally (reaching 95%+ on CIFAR-10 and 99%+ on PlantVillage), while validation loss exploded by an order of magnitude within 2–5 epochs (e.g., 1.66 → 13.41 on CIFAR-10 fold 0; 2.16 → 17.55 on PlantVillage fold 0). All runs triggered early stopping at epoch 8–9. This pattern is qualitatively distinct from classical overfitting and is analysed in detail in Section 5.6.

5.4 Statistical Significance

Table 6 presents the complete results of the 18 paired statistical tests.

Overall significance pattern. Of 18 paired comparisons, 15 reached statistical significance at $\alpha = 0.05$ (83%). All significant comparisons exhibited large Cohen's d values ($|d| > 3.0$ for accuracy/F1 metrics) and Cliff's delta of ± 1.0 (complete stochastic dominance). However, these statistics must be interpreted with caution: with only three paired observations per test, effect-size estimates are inflated by the narrow denominator. Under Bonferroni correction ($\alpha_{\text{adj}} = 0.05/18 \approx 0.0028$), only three comparisons survive: CIFAR-10 VGG16 throughput ($p = 0.005$), CIFAR-10 MobileNetV2 throughput ($p = 0.002$), and CIFAR-10 MobileNetV2 accuracy ($p = 0.008$). The PlantVillage ResNet50/VGG16 accuracy differences ($p \approx 0.02$–0.03) would no longer be significant, suggesting that their small absolute magnitudes (< 0.5 pp) may not generalise.

Non-significant cases. Three comparisons did not reach significance: CIFAR-10 ResNet50 accuracy ($p = 0.277$), CIFAR-10 ResNet50 macro-F1 ($p = 0.276$), and PlantVillage ResNet50 throughput ($p = 0.290$). The ResNet50 non-significance on CIFAR-10 is driven by PyTorch's high fold-level variance ($\sigma = 2.78\%$, reflecting the warmup instability in folds 0/1). Had all three PyTorch folds converged normally, PyTorch ResNet50

Table 6: Statistical Test Results (18 Paired Comparisons)

Dataset	Model	Metric	PT mean	Keras mean	Δ	p-value	Sig?	Cohen's d	Cliff's δ
PV	ResNet50	accuracy	0.9977	0.9947	+0.0030	0.021	Yes	3.90	1.00
PV	ResNet50	macro-F1	0.9964	0.9931	+0.0033	0.011	Yes	5.53	1.00
PV	ResNet50	img/sec	124.9	97.9	+27.0	0.290	No	0.82	1.00
PV	VGG16	accuracy	0.9976	0.9930	+0.0046	0.030	Yes	3.24	1.00
PV	VGG16	macro-F1	0.9966	0.9907	+0.0059	0.032	Yes	3.14	1.00
PV	VGG16	img/sec	76.3	72.8	+3.5	0.008	Yes	6.61	1.00
PV	MobNetV2	accuracy	0.9974	0.5718	+0.4256	0.019	Yes	4.09	1.00
PV	MobNetV2	macro-F1	0.9963	0.4688	+0.5275	0.013	Yes	5.06	1.00
PV	MobNetV2	img/sec	179.3	210.3	−31.0	0.029	Yes	−3.31	−0.56
C10	ResNet50	accuracy	0.9347	0.9585	−0.0239	0.277	No	−0.86	−0.33
C10	ResNet50	macro-F1	0.9345	0.9584	−0.0240	0.276	No	−0.86	−0.33
C10	ResNet50	img/sec	114.4	214.0	−99.6	0.013	Yes	−5.12	−1.00
C10	VGG16	accuracy	0.9573	0.9379	+0.0194	0.010	Yes	5.89	1.00
C10	VGG16	macro-F1	0.9572	0.9377	+0.0195	0.010	Yes	5.87	1.00
C10	VGG16	img/sec	65.6	137.1	−71.5	0.005	Yes	−7.99	−1.00
C10	MobNetV2	accuracy	0.9539	0.3961	+0.5578	0.008	Yes	6.25	1.00
C10	MobNetV2	macro-F1	0.9538	0.3757	+0.5781	0.011	Yes	5.57	1.00
C10	MobNetV2	img/sec	160.5	369.0	−208.6	0.002	Yes	−14.78	−1.00

PV = PlantVillage; C10 = CIFAR-10; MobNetV2 = MobileNetV2; NS at $\alpha = 0.05$.

would likely have matched or exceeded Keras; conversely, had only fold 0/1 results been observed, the conclusion would be that Keras substantially outperforms PyTorch. This underscores the importance of reporting per-fold results rather than only aggregates.

Practical vs. statistical significance. Statistical significance is dominated by two phenomena: (a) the catastrophic Keras MobileNetV2 failures (overwhelming effect, $\Delta > 42$ pp), and (b) smaller but consistent PyTorch advantages for ResNet50 and VGG16 on PlantVillage ($\Delta < 0.5$ pp) and VGG16 on CIFAR-10 ($\Delta = 1.94$ pp). When MobileNetV2 results are excluded, the accuracy differences between frameworks are modest ($<$ 2.5 pp) and their practical impact is limited. The most actionable finding is not the aggregate significance pattern, but rather the identification of specific (model, dataset) combinations where framework choice is critical (MobileNetV2) versus those where it is essentially neutral (ResNet50, VGG16 on PlantVillage).

5.5 VGG16-BN Cross-Framework Alignment Discussion

On PlantVillage, the 0.46 pp accuracy gap (Keras 99.30% vs. PyTorch 99.76%) is statistically significant ($p = 0.030$) but practically negligible. Both frameworks converged smoothly over 23–30 epochs, suggesting that on high-resolution, in-distribution data, the BN initialisation difference is absorbed during fine-tuning—23–30 epochs are sufficient for Keras BN to learn appropriate running statistics from the target domain.

On CIFAR-10, the gap widens to 1.94 pp (Keras 93.79% vs. PyTorch 95.73%, $p = 0.010$). The distribution shift (upsampled low-resolution images) amplifies the impact of BN initialisation: PyTorch's pretrained BN statistics provide a better starting point for feature normalisation under distribution-shifted conditions, while Keras BN must learn from scratch during a training process complicated by the non-standard input distribution.

The VGG16-BN comparison is therefore partially confounded by the BN initialisation asymmetry—an inherent limitation because `tf.keras.applications.VGG16` does not include BatchNorm. The accuracy differences cannot be attributed purely to framework choice but reflect a combination of framework-level and implementation-level factors. For tasks involving substantial distribution shift, frameworks providing pretrained BN statistics offer a measurable advantage. Future work should investigate whether custom Keras VGG16-BN models with converted pretrained BN statistics close this gap.

5.6 Keras MobileNetV2 Failure Analysis

5.6.1 Observed Phenomenon

Keras MobileNetV2 exhibited a consistent pathological pattern across all 6 runs (2 datasets $\times$ 3 folds). On CIFAR-10, training accuracy reached 95.2% by epoch 8, but validation loss exploded from 1.66 (epoch 1) to 13.41 (epoch 5). Validation accuracy collapsed to 16–48%. Test accuracies were: fold 0 = 47.77%, fold 1 = 40.87%, fold 2 = 30.21%. All folds early-stopped at epoch 8–9. On PlantVillage, training accuracy reached 99.2% by epoch 8, but validation loss exploded from 2.16 (epoch 1) to 17.55 (epoch 5). Validation accuracy collapsed to 3–66%. Test accuracies were: fold 0 = 45.56%, fold 1 = 65.81%, fold 2 = 60.18%. PyTorch MobileNetV2 under identical configuration

converged normally on both datasets—95.39% (CIFAR-10), 99.74% (PlantVillage)—with smooth training dynamics and no early stopping.

[Keras MobileNetV2 failure visualization] val_loss explosion vs. training accuracy annotated with LR at each epoch

Figure 3: Keras MobileNetV2 failure on CIFAR-10 fold 0: validation loss explodes while training accuracy continues to improve, indicating a train/validation distribution mismatch rather than classical overfitting.

5.6.2 Distinguishing Overfitting from Numerical Instability

The explosive validation-loss divergence (4–10× within 2–3 epochs) while training metrics remain healthy is qualitatively distinct from classical overfitting, in which validation loss increases gradually as the model memorises training data. In classical overfitting, training and validation accuracy diverge slowly, and the gap grows monotonically. In the observed failure, validation loss exhibits sudden, explosive growth while training accuracy continues to improve normally. This pattern indicates numerical instability or a severe feature-distribution mismatch between batch statistics (used during training in BatchNorm layers) and running statistics (used during validation and testing), rather than simple memorisation.

5.6.3 BatchNorm Momentum and Epsilon as the Primary Suspect

The most likely cause of this failure lies in the differing BatchNorm default parameters between frameworks. PyTorch uses momentum $= 0.1$ and $\varepsilon = 10^{-5}$, while Keras uses momentum $= 0.99$ and $\varepsilon = 10^{-3}$. The Keras convention defines momentum as the decay factor for the running mean (running_mean $\leftarrow$ momentum $\times$ running_mean $+$ $(1 - \text{momentum}) \times$ batch_mean), meaning that Keras updates running statistics with an effective rate of 0.01 per step, compared to PyTorch's 0.1. This 10× slower exponential moving average (EMA) update causes Keras running statistics to lag behind rapidly changing weight distributions under an aggressive learning-rate schedule. MobileNetV2's depthwise separable convolutions amplify per-channel normalisation errors: each depthwise filter operates on a single channel, providing no cross-channel averaging. Standard convolutions in ResNet50 and VGG16 mix information across channels, absorbing BN statistic mismatches through this implicit averaging.

5.6.4 Learning-Rate–BatchNorm Interaction

Epoch-level logs confirm that the monotonic validation-loss deterioration tracks the warmup learning-rate increase (LR $0.008 \rightarrow 0.04$, epochs 1–5). As the learning rate increases, weight magnitudes change rapidly, causing batch-level statistics to diverge from the slowly updating running statistics. During training, the model adapts to batch statistics and achieves high accuracy; during validation, the stale running statistics produce incorrect normalisation, leading to logit-scale distortion and explosive cross-entropy loss. The gradient landscape of Keras MobileNetV2 becomes unstable at elevated learning rates under these conditions, while PyTorch's faster running-statistic updates allow the model to tolerate the same schedule.

5.6.5 Cross-Dataset Consistency as Evidence of Systematic Cause

The identical failure pattern on both PlantVillage (high-resolution, in-distribution) and CIFAR-10 (low-resolution, distribution-shifted) eliminates dataset-specific artefacts as a plausible explanation. The failure is intrinsic to the Keras MobileNetV2 implementation under this hyperparameter regime. This cross-dataset consistency strengthens the conclusion that the root cause lies in the framework-level interaction between BatchNorm defaults and training dynamics, rather than in any property of the input data.

This case study elevates the paper's finding from "frameworks are roughly equivalent" to a more nuanced conclusion: framework equivalence is conditionally dependent on the (architecture, dataset, hyperparameter) triple. Framework comparison studies should not merely report average performance but must systematically identify failure modes—configurations where one framework produces catastrophically worse results. These failure modes constitute the most actionable information for practitioners, as they may be triggered silently in production deployments.

5.7 PyTorch ResNet50 on CIFAR-10: Warmup-Phase Instability

PyTorch ResNet50 on CIFAR-10 folds 0 and 1 early-stopped at epoch 8 with ∼91.9% accuracy. Validation accuracy peaked at epoch 1 during warmup (at LR ≈ 0.008), then declined monotonically as the learning rate increased toward the base rate of 0.04. Fold 2 converged normally to 96.69%

over 30 epochs. Keras ResNet50 showed no analogous instability—all three folds completed 30 epochs with stable convergence (95.70–95.96%).

The combination of AMP mixed precision, aggressive warmup LR (0.04), and distribution-shifted inputs creates a narrow stability region sensitive to classification-head initialisation. For folds 0 and 1, the random seed ($42 + \text{fold}$) placed the model in an unfavourable region where increasing LR during warmup pushed it away from the loss valley found at epoch 1. The practical implication is that for distribution-shifted transfer-learning tasks, a reduced base learning rate or extended warmup period is recommended as a precautionary measure.

5.8 Scenario-Specific Recommendations

Table 7 synthesises the experimental findings into deployment-oriented recommendations.

5.9 Limitations

Several limitations of this study should be acknowledged. First, all experiments were conducted on a single GPU (NVIDIA A40); results may differ on multi-GPU, TPU, or consumer-grade hardware configurations. Second, only SGD with Nesterov momentum was tested; Adam, AdamW, or LAMB optimizers may interact differently with framework-specific BatchNorm implementations. Third, CIFAR-10 bilinear upsampling ($32 \rightarrow 224$) introduces artefacts that interact non-trivially with pretrained models, limiting the generalisability of CIFAR-10-specific findings to native-resolution tasks. Fourth, the 3-fold cross-validation design yields only three paired observations per statistical test, providing limited statistical power; a 5- or 10-fold design would strengthen significance claims. Fifth, the VGG16-BN comparison is confounded by the BatchNorm initialisation asymmetry (pretrained vs. default), which is inherent to the framework APIs and cannot be eliminated without custom weight conversion. Sixth, Keras ran in float32 while PyTorch used AMP mixed precision (due to `LossScaleOptimizer` failure in custom `GradientTape` loops), introducing an asymmetry in both accuracy and throughput comparisons. Finally, no optimizer ablation was performed; framework-specific implementations of Adam or AdamW may exhibit different cross-framework behaviour.

6 CONCLUSION AND FUTURE WORK

This study systematically benchmarked Keras (TensorFlow backend) and PyTorch under strictly controlled conditions across three ImageNet-pretrained architectures (ResNet50, VGG16-BN, MobileNetV2), two datasets (PlantVillage and CIFAR-10), and three stratified cross-validation folds, yielding 36 training runs with every hyperparameter, data split, and augmentation matched. The central finding is that framework equivalence is *conditionally dependent* on the (architecture, dataset, hyperparameter) triple. For heavy architectures (ResNet50, VGG16), framework-induced accuracy differences were statistically significant in the majority of comparisons (15 out of 18 paired t-tests at $\alpha = 0.05$) but practically marginal (< 2.5 pp on both datasets); for these models, practitioners may choose either framework based on ecosystem preference, deployment infrastructure, or developer familiarity. However, for the lightweight architecture MobileNetV2, the Keras implementation exhibited catastrophic training instability across both datasets (6 out of 6 runs failed, accuracy degraded to 39–57% vs. PyTorch's 95–99%), demonstrating that framework choice can be a critical deployment variable for specific model families. The root cause was traced to differing BatchNorm momentum defaults (Keras 0.99 vs. PyTorch 0.1), which caused Keras running statistics to lag behind rapidly changing weight distributions under the aggressive cosine-annealing schedule—an effect amplified by MobileNetV2's depthwise separable convolutions. The throughput analysis revealed that neither framework is universally faster: Keras achieved 1.9–2.3$\times$ higher throughput on low-resolution upsampled data (CIFAR-10) due to `tf.data` pipeline efficiency, while PyTorch was comparable or slightly faster on native-resolution data (PlantVillage) due to AMP and optimised `DataLoader` configuration. Framework comparison studies should therefore not merely report aggregate performance metrics but must systematically identify and analyse failure modes—configurations where performance degrades catastrophically—as these represent the most actionable information for practitioners.

Several directions for future work are identified. First, the benchmark should be extended to additional architectures—including EfficientNet, Vision Transformer (ViT), and Swin Transformer—and additional datasets spanning medical imaging and satellite remote sensing to evaluate whether the conditional equivalence finding generalises across a broader range of model families and application domains. Second, alternative optimizers (Adam, AdamW, LAMB) should be tested to

Table 7: Scenario-Specific Recommendation Summary

Scenario	Recommended Config.	Rationale
High-res domain-specific (plant disease, medical)	Either framework + ResNet50 or VGG16	>99.3% on PlantVillage; framework guided by ecosystem preference
High-res + edge deployment (mobile, IoT)	PyTorch MobileNetV2	99.7% at 2.3M params; Keras MobileNetV2 unreliable
Low-res upsampled + stability required	Keras ResNet50	95.85% with $\sigma = 0.10\%$; most stable on CIFAR-10
Low-res + highest accuracy	PyTorch VGG16	95.73% on CIFAR-10; benefits from pretrained BN
Throughput priority (small source images)	Keras (any stable model)	1.9–2.3× faster on CIFAR-10 via `tf.data`
Throughput priority (native-res images)	Either framework	Difference small and dataset-dependent
Framework migration of lightweight models	Requires re-validation	Keras MobileNetV2 failure proves parity cannot be assumed

investigate optimizer–framework interactions, particularly whether adaptive-rate optimizers mitigate the BatchNorm momentum sensitivity observed with SGD. Third, the experimental protocol should be scaled to multi-GPU and distributed training settings to evaluate framework scaling efficiency and communication overhead. Fourth, a controlled study of BatchNorm hyperparameter alignment—explicitly setting identical momentum and epsilon values in both frameworks—would isolate the contribution of BN defaults from other framework-level differences. Fifth, the cross-validation design should be extended to 5 or 10 folds to increase the statistical power of paired tests and enable more robust effect-size estimation. Finally, open-set recognition and few-shot transfer-learning protocols should be investigated for practical deployment scenarios where the test distribution may not match the training distribution.

References

[1] Deng, J., Dong, W., Socher, R., Li, L., Li, K. and Fei-Fei, L., "ImageNet: A large-scale hierarchical image database," *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[2] Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H. and He, Q., "A comprehensive survey on transfer learning," *Proceedings of the IEEE*, Vol. 109, No. 1, 2021, pp. 43–76.

[3] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M. *et al.*, "TensorFlow: A system for large-scale machine learning," *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

[4] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. *et al.*, "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 8024–8035.

[5] He, K., Zhang, X., Ren, S. and Sun, J., "Deep residual learning for image recognition," *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[6] Simonyan, K. and Zisserman, A., "Very deep convolutional networks for large-scale image recognition," *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[7] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C., "MobileNetV2: Inverted residuals and linear bottlenecks," *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[8] Hughes, D.P. and Salathé, M., "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv preprint arXiv:1511.08060*, 2015.

[9] Mohanty, S.P., Hughes, D.P. and Salathé, M., "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, Vol. 7, 2016, Article 1419.

[10] Krizhevsky, A., "Learning multiple layers of features from tiny images," *Technical Report*, University of Toronto, 2009.

[11] Yosinski, J., Clune, J., Bengio, Y. and Lipson, H., "How transferable are features in deep neural networks?" *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 27, 2014, pp. 3320–3328.

[12] Shi, S., Wang, Q., Xu, P. and Chu, X., "Benchmarking state-of-the-art deep learning software tools," *Proc. 7th IEEE Int. Conf. Cloud Computing and Big Data Analysis*, 2016, pp. 99–104.

[13] Nguyen, G., Dlugolinsky, S., Bobák, M., Tran, V., López García, Á., Heredia, I., Malík, P. and Hluchý, L., "Machine learning and deep learning frameworks and libraries for large-scale data mining: A survey," *Artificial Intelligence Review*, Vol. 52, No. 1, 2019, pp. 77–124.

[14] Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D. and Meger, D., "Deep reinforcement learning that matters," *Proc. AAAI Conf. Artificial Intelligence*, Vol. 32, No. 1, 2018, pp. 3207–3214.

[15] Pham, H.V., Qian, S., Wang, J., Mou, T., Li, B., Zhang, L. and Li, J., "Problems and opportunities in training deep learning software systems: An analysis of variance," *Proc. 35th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2020, pp. 771–783.

[16] Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Mohammadi Sepahvand, N., Raff, E., Madan, K., Voleti, V., Ebrahimi Kahou, S. and Pal, C., "Accounting for variance in machine learning benchmarks," *Proc. Machine Learning and Systems (MLSys)*, Vol. 3, 2021, pp. 747–769.

[17] Chen, J., Zhang, D., Suzauddola, M. and Nanehkaran, Y.A., "Identification of plant disease images via a squeeze-and-excitation MobileNet model and twice transfer learning," *Scientific Reports*, Vol. 14, 2024, Article 12339.

[18] Dimitrovski, I., Kitanovski, I., Kocev, D. and Simidjievski, N., "Image classification of grape leaf diseases: Current state, challenges, and future directions," *Journal of Imaging*, Vol. 11, No. 7, 2025, Article 207.

[19] Shorten, C. and Khoshgoftaar, T.M., "A survey on image data augmentation for deep learning," *Journal of Big Data*, Vol. 6, No. 1, 2019, Article 60.

[20] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, Vol. 12, 2011, pp. 2825–2830.

[21] Ioffe, S. and Szegedy, C., "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[22] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. and Adam, H., "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.

[23] Loshchilov, I. and Hutter, F., "SGDR: Stochastic gradient descent with warm restarts," *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[24] Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y. and He, K., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," *arXiv preprint arXiv:1706.02677*, 2017.

[25] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G. and Wu, H., "Mixed precision training," *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[26] Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E. and Larochelle, H., "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, Vol. 22, No. 164, 2021, pp. 1–20.